DAA - Design and Analysis of Algorithms

Session 4 Lab Experiments

Name: Nithya Sree.S

Reg No: 192511058

Question 1: Positions of Large Groups

Problem: In a string s of lowercase letters, consecutive equal characters form groups. A group is identified by the interval [start, end] of its first and last index, and it is large if it has 3 or more characters. Return the intervals of every large group sorted in increasing order by start index.

Program (C):

```c
#include <stdio.h>
#include <string.h>

void largeGroups(char s[]) {
    int n = strlen(s), i = 0, start, found = 0;
    printf("[");
    while (i < n) {
        start = i;
        while (i < n && s[i] == s[start])
            i++;
        /* group is s[start..i-1] */
        if (i - start >= 3) {
            if (found == 1)
                printf(",");
            printf("[%d,%d]", start, i - 1);
            found = 1;
        }
    }
    printf("]\n");
}

int main() {
    printf("Input: s = \"abbxxxxzzy\"\nOutput: ");
    largeGroups("abbxxxxzzy");
    printf("\nInput: s = \"abc\"\nOutput: ");
    largeGroups("abc");
    printf("\nInput: s = \"abcdddeeeeaabbbcd\"\nOutput: ");
    largeGroups("abcdddeeeeaabbbcd");
    return 0;
}
```

Output:

Output - Program 1 (C)

```
$ gcc q1.c -o q1 && ./q1
Input: s = "abbxxxxzzy"
Output: [[3,6]]

Input: s = "abc"
Output: []

Input: s = "abcdddeeeeaabbbcd"
Output: [[3,5],[6,9],[12,14]]

=== Code Execution Successful ===
```

Complexity: Time: O(n) - the string is scanned once. Space: O(1) extra space.

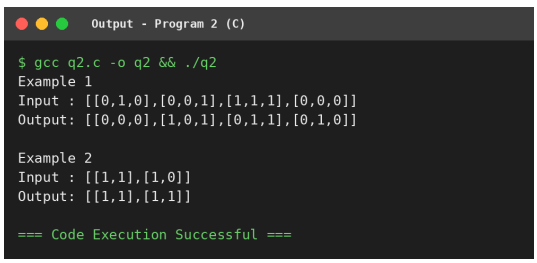
Question 2: Game of Life

Problem: The board is an m x n grid of cells, each live (1) or dead (0). Every cell interacts with its eight neighbours: a live cell with fewer than 2 or more than 3 live neighbours dies, a live cell with 2 or 3 live neighbours lives, and a dead cell with exactly 3 live neighbours becomes live. All changes happen at the same time. Given the current board, return the next state.

Program (C):

```
#include <stdio.h>
int countLive(int board[][10], int m, int n, int r, int c) {
    int i, j, count = 0;
    for (i = r - 1; i <= r + 1; i++) {
        for (j = c - 1; j <= c + 1; j++) {
            if (i == r && j == c)
                continue;
            if (i >= 0 && i < m && j >= 0 && j < n)
                count += board[i][j];
        }
    }
    return count;
}
void gameOfLife(int board[][10], int m, int n) {
    int next[10][10], i, j, live;
    for (i = 0; i < m; i++) {
        for (j = 0; j < n; j++) {
            live = countLive(board, m, n, i, j);
            if (board[i][j] == 1 && (live == 2 || live == 3))
                next[i][j] = 1;
            else if (board[i][j] == 0 && live == 3)
                next[i][j] = 1;
            else
                next[i][j] = 0;
        }
    }
    for (i = 0; i < m; i++)
        for (j = 0; j < n; j++)
            board[i][j] = next[i][j];
}
void printBoard(int board[][10], int m, int n) {
    int i, j;
    printf("[");
    for (i = 0; i < m; i++) {
        printf(i == 0 ? "[" : ",[");
        for (j = 0; j < n; j++)
            printf(j == 0 ? "%d" : ",%d", board[i][j]);
        printf("]");
    }
    printf("]\n");
}
void run(int board[][10], int m, int n) {
    printf("Input : ");
    printBoard(board, m, n);
    gameOfLife(board, m, n);
    printf("Output: ");
    printBoard(board, m, n);
}
int main() {
    int b1[10][10] = {{0, 1, 0}, {0, 0, 1}, {1, 1, 1}, {0, 0, 0}};
    int b2[10][10] = {{1, 1}, {1, 0}};
    printf("Example 1\n");
    run(b1, 4, 3);
    printf("\nExample 2\n");
    run(b2, 2, 2);
    return 0;
}
```

Output:



Complexity: Time: O(m x n x 8) = O(m x n). Space: O(m x n) for the next-state board.

Question 3: Champagne Tower

Problem: Glasses are stacked in a pyramid (row i has i + 1 glasses) and each glass holds one cup. Champagne is poured into the top glass; any excess falls equally to the glass on its left and right below. Given the number of cups poured, return how full the jth glass in the ith row is (both 0-indexed).

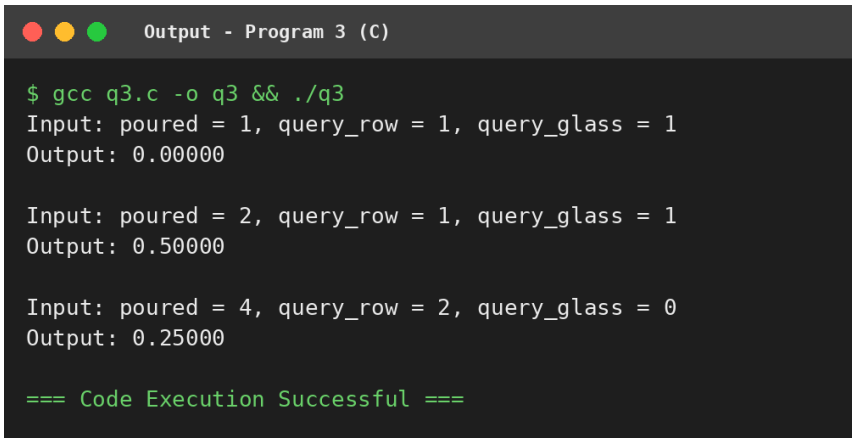
Program (C):

```c
#include <stdio.h>

double champagneTower(int poured, int query_row, int query_glass) {
    double glass[102][102] = {0};
    int r, c;
    double extra;
    glass[0][0] = poured;
    for (r = 0; r <= query_row; r++) {
        for (c = 0; c <= r; c++) {
            extra = (glass[r][c] - 1) / 2.0;     /* excess split to left and right */
            if (extra > 0) {
                glass[r + 1][c] += extra;
                glass[r + 1][c + 1] += extra;
            }
        }
    }
    if (glass[query_row][query_glass] > 1)
        return 1;
    return glass[query_row][query_glass];
}

int main() {
    printf("Input: poured = 1, query_row = 1, query_glass = 1\n");
    printf("Output: %.5f\n\n", champagneTower(1, 1, 1));
    printf("Input: poured = 2, query_row = 1, query_glass = 1\n");
    printf("Output: %.5f\n\n", champagneTower(2, 1, 1));
    printf("Input: poured = 4, query_row = 2, query_glass = 0\n");
    printf("Output: %.5f\n", champagneTower(4, 2, 0));
    return 0;
}
```

Output:



Complexity: Time: O(row^2) - every glass up to the query row is visited once. Space: O(row^2) for the glass table.